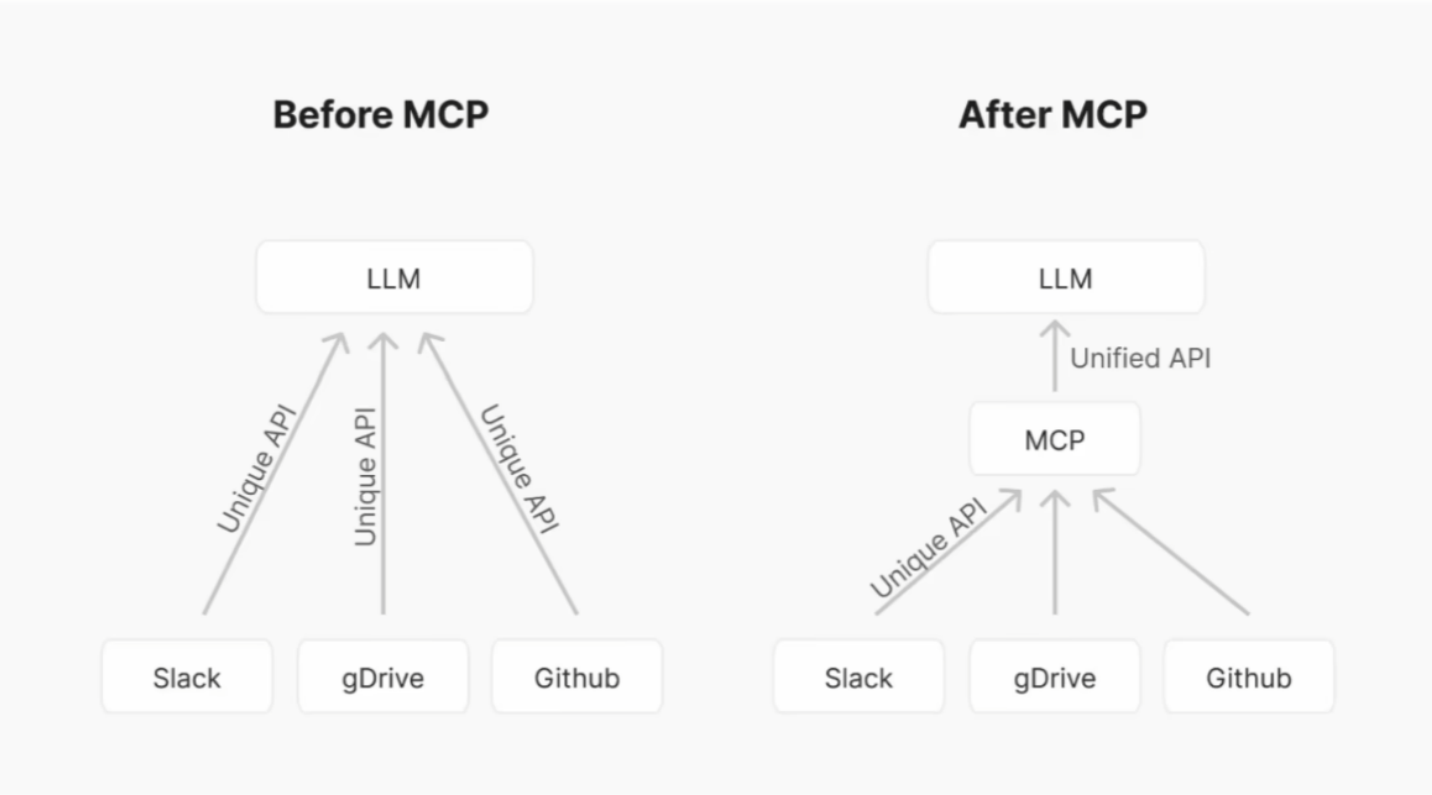


MCP 副本

什么是MCP

MCP（Model Context Protocol，模型上下文协议）是由 Anthropic 公司于 2024 年 11 月开源的标准化协议框架，旨在统一 AI 模型与外部数据源、工具及系统的交互方式，被誉为“AI 界的 USB-C 接口”。其核心目标是通过标准化协议打破数据孤岛，让 AI 应用能以一致的方式连接各类资源，实现更高效的集成与协作。



MCP有哪些部分

MCP 采用客户端 - 服务器（C/S）架构，包含三个关键组件：

1. MCP 主机（Host）

搭载 AI 智能体的应用系统（如聊天工具、IDE），负责发起请求并协调交互流程。

1. 多服务器连接与动态能力交换

- 连接管理：Host 可同时连接多个 MCP Server，每个 Server 提供特定功能（如天气查询、数据库操作）。例如，Claude Desktop 通过 MCP Host 连接本地文件服务器和远程 API 服务，实现文档分析与实时数据整合。

- 能力协商：MCP Client 与 Server 在建立连接时会交换“能力声明”（如支持的工具类型、数据格式），Host 根据任务需求动态选择最优服务。例如，AI 代码助手可自动调用 Git Server 进行版本控制，或调用数据库 Server 查询数据。

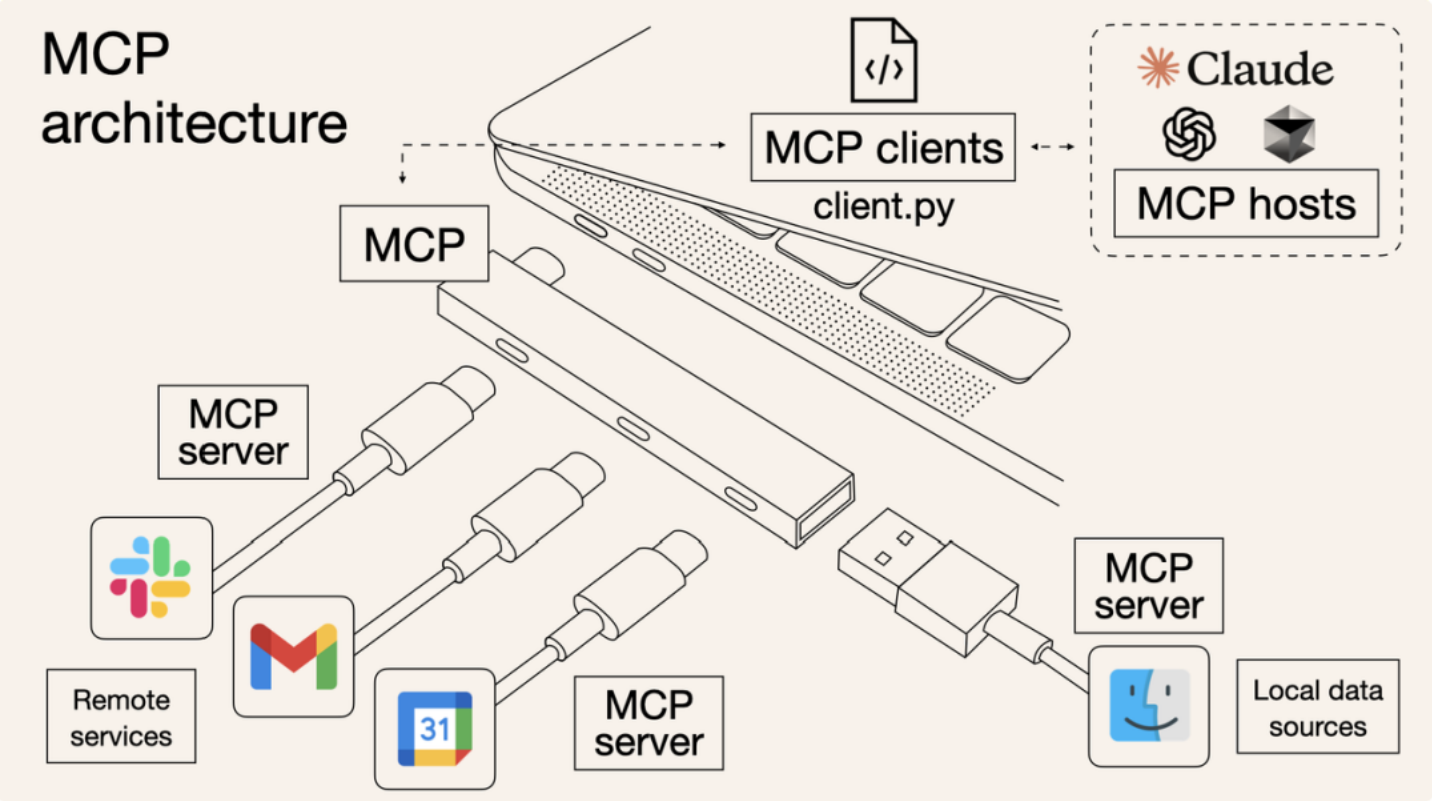
2. 安全的数据访问机制

- 本地与远程资源整合：Host 通过 MCP 协议安全访问本地数据源（如计算机文件、私有数据库）和远程服务（如第三方 API）。敏感数据（如银行账户）仅在本地 Server 处理，AI 模型仅获取处理结果，避免原始数据泄露。
- 权限管控：支持分级权限管理（如管理员 / 普通用户），结合传输加密（TLS）和访问审计，确保数据安全。例如，企业内部 AI 工具可限制员工对客户数据库的访问权限。

3. 统一的用户交互界面

- 任务调度与结果整合：Host 将用户请求转化为标准化协议消息，协调多个 Server 协作完成任务，并将结果整合为自然语言响应。例如，用户要求“分析财务报表并生成投资建议”，Host 会调用文件 Server 读取报表、数据库 Server 获取市场数据，最终由 AI 模型综合输出建议。
- 流式交互支持：支持多轮对话中的上下文记忆（如持续跟进旅行计划）和断网续传功能，提升复杂场景下的用户体验。

三、典型应用场景



1. AI 开发与集成

- 工具链整合：开发者通过 MCP Host 将 AI 模型与代码编辑器（如 VS Code）、版本控制系统（Git）、调试工具等集成，实现自动化代码生成、测试用例生成等功能。例如，华为云开发者空间通过 MCP Host 调用 DeepSeek 模型，结合自定义 Server 实现智能代码补全。
- 多模态交互：Host 可连接视觉、语音等多模态 Server，扩展 AI 应用能力。例如，输入图片描述后，Host 调用图像识别 Server 获取内容，再由语言模型生成相关故事。

2. 企业级 AI 部署

- 数据合规性管理：企业通过 MCP Host 将 AI 模型与内部系统（如 ERP、CRM）连接，确保数据仅在授权范围内使用。例如，医疗 AI 工具通过 MCP 协议访问患者病历，严格遵循 HIPAA 等隐私法规。
- 跨平台协作：支持混合云部署，Host 可同时连接私有云 Server（处理敏感数据）和公有云 Server（调用通用工具），实现灵活的资源调度。例如，金融机构的风险评估系统可调用本地风控模型和云端市场数据 API。

3. 物联网与边缘计算

- 设备管理：在物联网场景中，MCP Host 可作为边缘节点，连接传感器、执行器等设备的 MCP Server，实现实时数据采集与控制。例如，智能工厂通过 Host 协调机器人控制 Server 和环境监测 Server，优化生产流程。
- 大规模连接优化：腾讯云 MCP 物联网平台通过分布式 Host 集群管理百万级设备连接，结合动态资源分配算法，提升数据处理效率并降低延迟。

四、技术优势与行业价值

1. 标准化与互操作性

MCP 协议统一了 AI 模型与外部资源的交互方式，避免重复开发适配接口。例如，开发者无需为每个工具单独编写 API 客户端，只需通过 MCP Host 调用标准化服务，开发效率提升 50% 以上。

2. 扩展性与灵活性

支持动态加载新 Server 或协议插件，快速适配新型设备或业务需求。例如，自定义协议适配器可使 MCP Host 兼容二进制传感器数据格式，而无需修改核心代码。

3. 性能优化

通过 KV-Cache 复用历史计算结果、异步并行通信等技术，Host 在处理高并发请求时响应速度提升 40% 以上。例如，大型客服系统通过 MCP Host 调用知识库 Server，实现毫秒级响应。

4. 生态

目前已有数千种 MCP Server 工具可供直接使用（如天气查询、邮件管理、Git 仓库操作），形成繁荣的开发者社区。华为云、腾讯云等平台提供 MCP 集成解决方案，降低企业 AI 落地门槛。

2. MCP 客户端（Client）

作为 Host 内部的接口层，管理与 MCP 服务器的一对一连接，处理协议转换和通信标准化。

核心功能

1. 初始化与连接管理

- 创建客户端实例并连接到 MCP 服务器
- 支持自定义传输协议（HTTP/WebSocket）和认证方式

2. 工具调用与参数验证

- 根据服务器提供的工具清单，构造合法请求
- 支持同步 / 异步调用，处理返回结果

3. 流式响应处理

- 实时接收和解析服务器返回的数据流
- 支持文本、JSON、二进制等多种格式

4. 认证与权限管理

- 处理 OAuth2.1 认证流程
- 自动刷新访问令牌，维护会话状态

代码块

```
1  from mcp.client import McpClient, McpToolCall
2  from mcp.auth import OAuth2Authenticator
3  import asyncio
4
5  # 初始化带有认证的客户端
6  async def create_authenticated_client():
7      auth = OAuth2Authenticator(
8          client_id="your_client_id",
9          client_secret="your_client_secret",
10         token_endpoint="https://auth.example.com/token",
11         scopes=["mcp:tools"]
12     )
13
14     client = McpClient(
15         server_url="https://mcp-server.example.com",
16         authenticator=auth
17     )
18
19     # 连接服务器并获取工具清单
20     await client.connect()
21     tools = await client.get_tools()
22     print(f"可用工具: {[tool.name for tool in tools]}")
23     return client
24
```

```
25 # 调用数据库查询工具 (异步示例)
26 async def call_database_tool():
27     client = await create_authenticated_client()
28
29     # 构造工具调用参数
30     tool_call = McpToolCall(
31         name="database_query",
32         parameters={
33             "query": "SELECT * FROM users LIMIT 10",
34             "connection_id": "prod-db-1"
35         }
36     )
37
38     # 发送工具调用请求 (支持流式响应)
39     async for chunk in client.stream_call(tool_call):
40         if chunk.is_final:
41             print(f"查询结果: {chunk.content}")
42         else:
43             print(f"部分结果: {chunk.content}")
44
45     await client.disconnect()
46
47 # 同步方式调用文件操作工具
48 def call_file_tool():
49     # 创建同步客户端
50     with McpClient("https://mcp-server.example.com") as client:
51         # 上传文件
52         with open("data.csv", "rb") as file:
53             result = client.call_tool(
54                 name="file_upload",
55                 parameters={
56                     "path": "/uploads/data.csv",
57                     "content": file.read()
58                 }
59             )
60             print(f"上传结果: {result.content}")
61
62 # 主函数
63 if __name__ == "__main__":
64     # 运行异步示例
65     asyncio.run(call_database_tool())
66
67     # 运行同步示例
68     call_file_tool()
```

3. MCP 服务器 (Server)

通过标准化协议提供特定能力的轻量级程序，可访问本地文件、数据库、API 等资源，或执行工具调用（如发送邮件、生成图像）。

1. 资源与工具管理

- 支持通过 URI 模式暴露文件、数据库记录等资源实体，并提供动态变更通知机制，确保客户端实时获取更新。
- 允许将文件读写、API 调用等功能封装为工具，供 AI 模型直接调用，实现复杂任务自动化（如自然语言操作本地数据库）。

2. 协议交互与会话控制

- 基于 JSON-RPC 2.0 协议处理客户端请求，支持同步 / 异步 API 及能力协商，可灵活集成到 Spring 等主流框架。
- 管理多客户端并发连接，提供 SSE（服务器端事件流）、STDIO 等多种传输方式，确保通信高效稳定。

3. 开发与运维支持

- 提供结构化日志系统，支持不同级别日志消息推送，方便调试与监控。
- 支持自定义提示模板，优化 AI 模型的上下文理解能力，提升响应准确性。

典型应用场景包括 AI 驱动的 UI 自动化（如通过 Playwright 操作浏览器）、本地资源智能管理（如文件 / 数据库操作）等。

代码块

```
1  import asyncio
2  import mcp.types as types
3  from mcp.server import Server
4  from mcp.server.stdio import stdio_server
5
6  app = Server("example-server")
7
8  @app.list_resources()
9  async def list_resources() -> list[types.Resource]:
10     return [
11         types.Resource(
12             uri="example://resource",
13             name="Example Resource"
14         )
15     ]
16
17  async def main():
18     async with stdio_server() as streams:
```

```

19         await app.run(
20             streams[0],
21             streams[1],
22             app.create_initialization_options()
23         )
24
25     if __name__ == "__main__":
26         asyncio.run(main())

```

MCP有什么作用

1. 统一工具调用标准

传统 AI 模型的 Function Call 接口碎片化严重，开发者需为不同模型定制工具集成。MCP 整合了主流模型的工具调用标准，提供统一的协议规范，显著降低开发复杂度。例如，开发者只需编写一次 MCP Server，即可被 Claude、ChatGPT 等多个模型调用。

2. 动态上下文管理

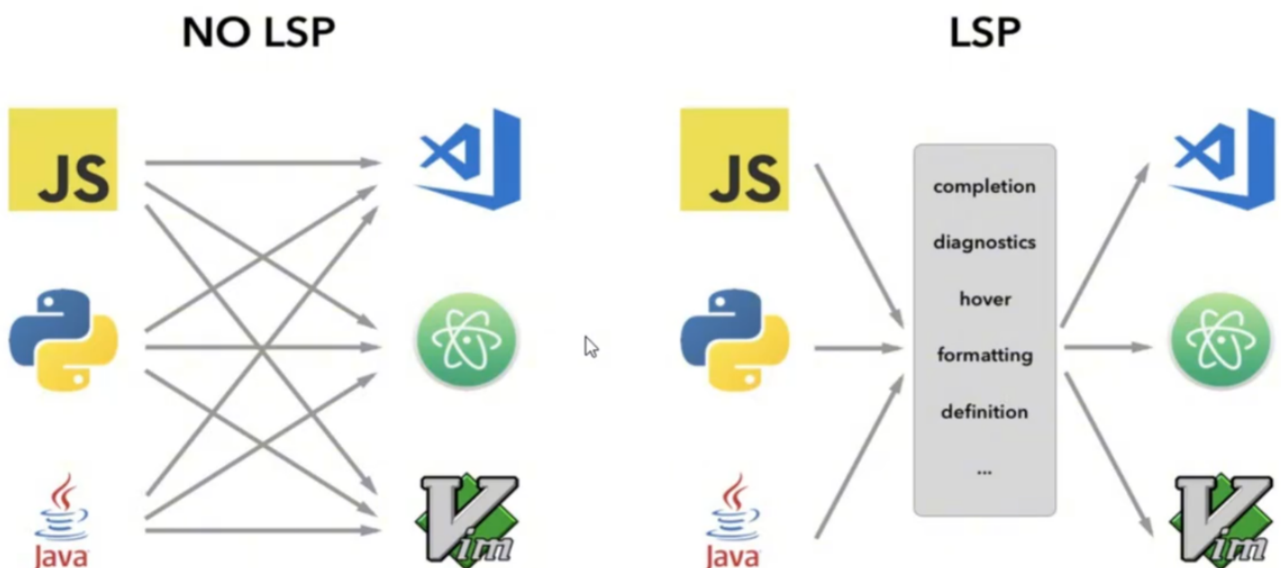
MCP 支持持久化上下文窗口，可存储用户偏好、会话历史和环境数据。AI 模型能基于上下文调整策略，实现多步骤任务的连贯性执行（如“研究→起草→编辑→发布”博客文章）。例如，在医疗场景中，MCP 可记录患者过敏史，辅助模型动态调整诊断建议。

3. 即插即用的生态扩展

MCP Server 可动态接入各类工具，形成开放生态。例如，Smithery.ai 平台已收录超过 4,219 种 MCP Server，涵盖 GitHub 交互、浏览器内容读取、图像生成等功能。开发者通过“插拔” MCP Server，可快速扩展 AI 应用的能力边界。

4. 增强安全性与可审计性

MCP 提供标准化的安全协议，支持数据加密和访问控制。同时，完整的上下文历史记录可追溯 AI 决策过程，满足合规性要求，尤其适用于医疗、金融等敏感领域。



MCP和function calling的区别

一、相同点

- 1. 服务于模型能力扩展
均为增强模型功能的机制，帮助模型处理复杂任务或上下文信息。
- 2. 数据交互本质
涉及数据的传递与处理（如输入参数、输出结果、上下文状态）。
- 3. 依赖协议 / 接口
需要定义明确的格式或规范（如 JSON Schema、函数签名）以实现交互。

二、不同点

维度	MCP（模型上下文协议）	函数调用（Function Call）
核心目标	管理模型上下文（如对话历史、参数状态、语境理解）	调用外部工具 / 函数执行具体操作（如计算、查询）
作用范围	模型内部或模型与外部环境的上下文传递（如多轮对话）	模型与外部工具 / 服务的功能交互（跨系统调用）
交互方式	基于上下文协议传递状态数据（如对话轮次、用户意图）	基于函数接口传递参数并触发执行（如 API 调用）
数据类型	上下文元数据（如角色标识、对话阶段、历史记录摘要）	具体任务参数（如数值、字符串、结构化数据）
执行时机	贯穿模型处理全流程（如每轮对话前更新上下文）	按需触发（如模型判断需要工具辅助时调用）
返回结果	上下文状态更新（如追加对话记录、更新用户画像）	具体任务结果（如计算值、查询数据、文件流）
技术实现	依赖内存存储、序列化协议（如 JSON、ProtoBuf）	依赖 RPC、HTTP 等通信协议或语言接口（如 Python API）

三、优势对比

MCP 的优势

- 1. 上下文连贯性
确保多轮交互中模型对用户意图的持续理解（如对话机器人的上下文记忆）。
- 2. 状态管理灵活性
支持动态调整上下文权重（如优先处理最新对话、区分不同用户会话）。
- 3. 轻量级交互
仅传递上下文元数据，无需执行实际计算，资源开销低。

函数调用的优势

- 1. 能力扩展性
突破模型内置功能限制（如调用实时天气 API、数据库查询工具）。
- 2. 结果精确性
利用专业工具实现高精度计算（如数学公式求解、代码执行）。
- 3. 实时数据处理
获取外部动态信息（如股票行情、新闻资讯），增强回答时效性。

四、劣势对比

MCP 的劣势

- 1. 上下文容量限制
受模型最大输入长度限制（如 GPT-4 支持 8k/32k tokens），长对话可能丢失早期信息。
- 2. 语义理解依赖模型
若上下文解析错误（如歧义意图），可能导致后续响应偏差。
- 3. 缺乏实际操作能力
仅管理状态，无法执行具体任务（如数据查询、文件生成）。

函数调用的劣势

- 1. 调用延迟与可靠性
依赖网络或外部服务稳定性，可能引入延迟或调用失败（如 API 超时）。
- 2. 参数解析复杂度
需确保模型生成的函数参数格式正确（如 JSON 格式错误会导致调用失败）。
- 3. 过度依赖外部工具
若工具不可用，模型功能会受限于自身内置能力。

五、典型应用场景

- MCP 适用场景：
 - 对话系统（如客服机器人、智能助手）的上下文管理；

- 多轮交互任务（如问卷调查、流程引导）的状态跟踪；
- 模型微调或提示工程中的上下文优化（如传递用户偏好参数）。
- 函数调用适用场景：
 - 需要实时数据的场景（如天气查询、航班动态）；
 - 复杂计算任务（如财务报表分析、科学建模）；
 - 操作外部资源（如文件读写、数据库增删改查）。

六、协同使用场景

在实际 AI 应用中，MCP 与函数调用常结合使用：

1. 对话 + 工具调用：

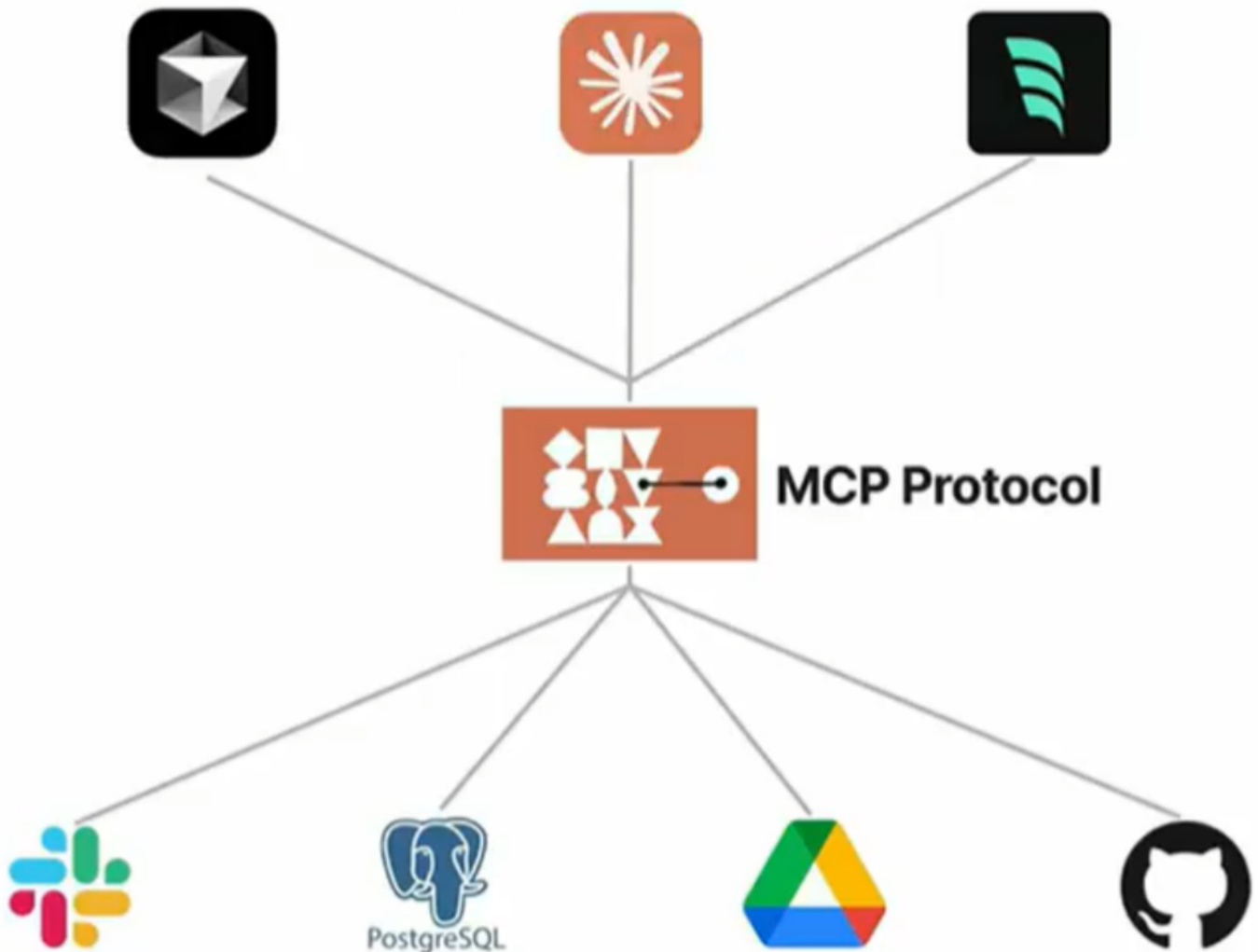
- MCP 管理对话上下文（如用户历史问题、身份信息）；
- 函数调用触发工具获取实时数据（如根据用户所在地查询天气），结果再通过 MCP 整合到后续回答中。

2. 多步任务处理：

- MCP 记录任务阶段（如 “用户已提供地址，下一步需确认联系方式” ）；
- 函数调用执行各阶段操作（如调用短信接口发送验证码）。

配置MCP的常见方法

With MCP



MCP（模型上下文协议）的三种实现方式——原生 IDE（如 Cursor）、插件（如 Cline）和独立软件（如 CherryStudio）——在技术架构、使用场景和生态整合上各有侧重，以下从核心优势、潜在局限及适用场景三方面展开分析：

一、原生 IDE（以 Cursor 为例）

核心优势：

1. 深度集成与无缝协作

原生 IDE 将 MCP 功能直接嵌入开发环境，与代码编辑、调试、版本控制等核心功能深度耦合。例如 Cursor 通过 MCP 协议直接访问文件系统、浏览器调试工具和终端，实现“AI 辅助开发”全流程闭环。开发者无需切换工具，可在同一界面完成代码生成、错误诊断和实时测试，显著提升效率。

2. 高性能与稳定性

原生实现通常经过 IDE 厂商的优化，能更好地利用底层资源。例如 Cursor 的 MCP 客户端与 IDE 内核共享内存空间，避免插件式方案常见的进程间通信开销，尤其在处理大规模代码分析或实时协作时表现更优。

3. 统一用户体验

从界面设计到操作逻辑均保持一致性，用户无需学习额外交互模式。例如 Cursor 的 AI 助手与代码补全功能采用相同快捷键和上下文菜单，降低认知成本。

潜在局限：

1. 定制化能力受限

功能扩展依赖 IDE 厂商的更新计划，开发者难以按需调整 MCP 的实现细节。例如 Cursor 的 MCP 服务器配置选项较少，无法直接对接某些私有数据源。

2. 跨平台兼容性

部分原生 IDE 仅支持特定操作系统（如 Cursor 早期版本仅限 macOS），且不同平台间的 MCP 功能可能存在差异。

适用场景：

- 复杂项目开发：需要 AI 深度参与代码生成、调试和重构的场景，如全栈应用开发。
- 团队协作：依赖 IDE 内置协作功能（如代码评审、版本控制）与 MCP 结合的场景，如敏捷开发团队。

二、插件（以 Cline 为例）

核心优势：

1. 灵活扩展与轻量化部署

插件通过标准化接口（如 VS Code 的 API）接入 MCP，可独立于 IDE 更新。例如 Cline 作为 VS Code 插件，用户可自由选择是否启用 MCP 功能，且不影响 IDE 原有性能。

2. 多 IDE 兼容性

同一插件可适配多种开发环境。例如 Cline 不仅支持 VS Code，还计划推出 IntelliJ 插件，满足不同开发者的工具偏好。

3. 社区驱动与快速迭代

开源插件（如 Cline）通常由社区维护，功能更新响应迅速。例如 Cline 在 2025 年 3 月的版本中新增对 MCP 协议 2.0 的支持，而 VS Code 官方尚未完全集成该功能。

潜在局限：

1. 性能瓶颈

插件依赖 IDE 的进程间通信机制，在处理高频工具调用（如实时代码分析）时可能出现延迟。例如 Cline 在调用外部 MCP 服务器时，响应时间比 Cursor 原生实现慢 10-15%。

2. 功能碎片化

插件开发者需自行处理与 IDE 其他功能的兼容性问题。例如 Cline 的文件操作工具需单独实现与 VS Code 文件系统的对接，可能出现权限冲突。

适用场景：

- 个性化开发需求：需要定制 MCP 工具链（如对接私有 API）的场景，如企业内部开发。
- 跨工具协作：需在不同 IDE 中复用 MCP 功能的场景，如自由职业者切换开发环境。

三、独立软件（以 CherryStudio 为例）

核心优势：

1. 专注 AI 能力与跨平台支持

独立软件通常围绕 MCP 协议设计，提供更专业的 AI 功能。例如 CherryStudio 内置 DeepSeek-V3 模型，支持多模态交互（文本、图像、Office 文件），且兼容 Windows、macOS 和 Linux。

2. 数据安全与隐私保护

独立软件可在本地运行 MCP 服务器，避免云端传输敏感数据。例如 CherryStudio 的文件系统 MCP 服务器支持本地加密，适合处理企业机密代码。

3. 低门槛上手

无需依赖复杂 IDE 配置，开箱即用。例如 CherryStudio 的 AI 对话框支持自然语言生成代码，新手开发者可快速上手。

潜在局限：

1. 与 IDE 集成度低

独立软件通常无法直接调用 IDE 的底层功能（如调试器），需通过文件同步或 API 间接交互。例如 CherryStudio 生成的代码需手动导入 VS Code 进行调试。

2. 生态扩展性弱

第三方工具支持有限。例如 CherryStudio 的 MCP 市场仅提供 300 + 预配置助手，远少于 Cursor 和 Cline 的社区资源。

适用场景：

- 轻量级开发任务：简单脚本编写、原型设计等无需复杂 IDE 功能的场景。
- 数据敏感场景：需本地处理代码或文档的场景，如金融行业的内部工具开发。

四、综合对比与选择建议

维度	原生 IDE	插件	独立软件
集成深度	最高（与 IDE 功能无缝衔接）	中等（依赖 IDE API）	最低（需手动同步数据）
性能表现	优（底层优化）	中（进程间通信开销）	良（专注 AI 模块）
灵活性	低（依赖厂商更新）	高（社区驱动）	中（可配置但扩展性有限）
适用场景	复杂项目开发、团队协作	个性化需求、跨 IDE 支持	轻量任务、数据敏感场景
典型工具	Cursor	Cline	CherryStudio